

The Alberta Buck - Notes Flow (DRAFT v0.6)

How One BUCK Transfer Becomes N Unlinkable Notes (Step by Step)

Perry Kundert

2026-04-22



This document is the *narrative* companion to alberta-buck-notes.org (architectural overview), alberta-buck-proofs.org (formal correctness), and alberta-buck-ethereum.org (Solidity surface). It walks one concrete example – Alice mints a 1,200-BUCK transfer into six notes of denominations [1000, 100, 25, 25, 25, 25] – end-to-end through the **Phase-7-bis SNARK-verified batch-mint pipeline**, and then walks the *inverse* flow: Bob redeems his 1,000-BUCK note via the Phase-7 spend circuit, receives 1,000 BUCK from the pool via a demurrage-carrying transfer, and leaves no chain-observable link back to `cm_0` (the specific commitment he opened). Every actor, every artifact, and every check is named.

The mint pipeline is in the middle of a planned pivot away from the original Phase-7 design (per-leaf on-chain Poseidon insertion at ~800K gas/leaf, capped by the block limit at $N \leq \sim 30$ in practice). In the new design the per-leaf Merkle-tree insertions are pushed *into* the mint SNARK: the circuit takes `oldRoot` as a public input, folds N commitments into the rolling tree using witness sibling paths, and emits a SNARK-attested `newRoot` for the contract to record. On-chain mint cost becomes essentially constant in N – ~380K plus ~500 gas per leaf of calldata – making honest single-call batches of 16, 128, or 1024 notes practical. The full design memo, including R1CS counts, gas projections, and a per-attack-vector hostile-Alice analysis, is alberta-buck-notes-rollup-mint.org.

The Phase-7 spend pipeline shipped in the bearer-shape (`spend.circom`, `SpendGroth16Verifier`, `Notes.spend`) and is exercised end-to-end in `test/SpendVerifier.t.sol`; the A-flavor in-circuit ElGamal/Chaum-Pedersen variant remains deferred to a future `spend_a.circom`, and that gap is flagged inline rather than papered over. The spend path is *unchanged* by the mint pivot: it already takes `cm` as a private witness under a public `noteRoot`, and the rolling-tree-update trick is contained to `mint`.

The aim is to show, by walking the path, that the system delivers five concrete guarantees:

1. **Atomic conservation** – if the transaction succeeds, the BUCK balance leaves Alice exactly once and matches the public total of the new notes. If the proof fails, no BUCK moves and no commitments are appended.
2. **Issuer-bound, recipient-blinded artifacts** – the public commitments name the issuer (`msg.sender`) but reveal nothing about per-note values or recipients.
3. **Per-note disclosure to recipients only** – each recipient receives an *opening* off-chain that lets them reconstruct exactly their own note’s value and the issuer’s verifiable identity, plus the fact that N siblings exist whose hidden values sum to `totalFace` – and nothing more.
4. **Offline receipt reconstruction** – given only the mint transaction ID, anyone holding an opening can independently verify the note against the chain, with no help from Alice or any indexer.
5. **Adversarial-prover soundness** – even if Alice runs a maliciously patched EVM and an arbitrarily hostile witness generator, the honest validators executing the deployed `Notes.sol` and `MintGroth16Verifier.sol` reject every batch that does not satisfy the mint relation. The on-chain truth is what governs.

The remaining caveats (the deferred A-flavor in-circuit identity binding, and the pinned- N dispatch set under `mint_batch.circom`) are addressed at the end and cross-referenced into the proofs and implementation docs. The mint-circuit field-overflow concern from earlier drafts has been closed by the `Num2Bits(128)` range bounds in `mint_batch.circom` and the matching bounds in `spend.circom`.

Contents

1	Cast of Characters	5
2	The Promise	5
3	What’s Inside a Note (Conceptually)	6
4	The Mint, Step by Step	6
4.1	Step 0: Pre-flight (Off-Chain, In Alice’s Wallet)	6
4.2	Step 1: Build Commitments	7
4.3	Step 2: Generate the Batch-Mint Groth16 Proof	7
4.4	Step 3: Submit <code>Notes.mint</code> On Chain	8
4.5	Step 4: Off-Chain Note Delivery	10
4.6	Step 5: Recipient Verification (Offline, Independent)	10
5	Why Alice Cannot Cheat (Even With a Hostile EVM and Prover)	11
5.1	Pillar 1: The On-Chain Verifier Is Fixed Code	11
5.2	Pillar 2: BUCK Escrow Is Inside the Same Atomic Call	12
5.3	Pillar 3: Public Inputs Are Sealed Into the Proof	12
5.4	Putting the Pillars Together: The Adversarial Walk	13
6	Why Outside Observers Cannot Link	14
6.1	Spend-Time Unlinkability	15
7	Atomicity, No Double-Mint, No Replay	15
7.1	Atomicity	15
7.2	No Double-Mint From One Transfer	15
7.3	No Replay	15
7.4	Cross-Chain or Cross-Deployment Replay	16
8	Prover Contention and the Rollup Race	16
8.1	Who Wins When Two Minters Race?	16
8.2	Mitigation Layers	17
8.3	What Does Not Regress	17
9	Receipt Construction From the Transaction ID Alone	17
10	The Redemption (Spend), Step by Step	18
10.1	Step 0: Bob Holds a Verified Note	19
10.2	Step 1: Fetch the Commitment Tree and Build a Merkle Path	19
10.3	Step 2: Compute the Nullifier	19
10.4	Step 3: Generate the Spend Proof	20
10.5	Step 4: Submit <code>Notes.spend</code> On Chain	21
10.6	Step 5: BUCK ERC-20 Pays Out (<code>transferCarrying</code>)	22
10.7	What the Observer Sees at Spend (Mallory Returns)	23
10.8	Why Alice Cannot Steal Bob’s Note (Even Between Rooms)	23
10.9	B-Flavor Notes Today	24
10.10	Recapping Property C (Only Valid Notes Redeem)	24

11 Demurrage-Carrying Transfers	25
11.1 Two Ways to Handle Demurrage at Transfer Time	25
11.2 Why the Note Pool Needs It	25
11.3 Shipped Surface in <code>Buck.sol</code>	26
11.4 Who Else Might Call It	27
11.5 Open Question: Identity Receipts on Carry	27
12 Putting the Guarantees Side-by-Side	27
13 Open Questions and Forward-Looking Caveats	28
14 Cross-References	28

DRAFT

1 Cast of Characters

The example uses concrete names so the message flow is easy to track:

- **Alice** – the issuer. She holds 1,200 BUCK and wants to break it into six notes. She is registered with the IdentityRegistry so that her `msg.sender` address resolves to a cryptographically attested identity.
- **Bob, Carol, Dave, Eve, Frank, Grace** – six recipients. Bob will get the 1,000-BUCK note; Carol the 100-BUCK note; Dave/Eve/Frank/Grace one 25-BUCK note each. Recipients do *not* need to be on-chain known to Alice – the recipient identity binding is a field inside the commitment, not an on-chain registration step at mint.
- **The honest EVM** – every Ethereum validator running stock client software. This is the only EVM whose verdict matters. It executes the deployed `Notes.sol`, `MintGroth16Verifier.sol`, `MintVerifierAdapter.sol`, and `Buck.sol` bytecode.
- **Alice’s hostile EVM (Eve-EVM)** – a fictional entity Alice can run locally. She may patch any contract logic she wants, run any witness generator, and produce any proof bytes. The point of the design is that nothing she runs locally affects what the honest EVM accepts.
- **A passive observer (Mallory-the-watcher)** – someone scanning the public chain after the fact, trying to learn *which* note went to *whom* for *how much*. Mallory has full chain history and unlimited compute.

2 The Promise

By the end of the mint transaction Alice will have:

- A single Ethereum transaction `T_mint` on-chain whose effects are:
 - 1,200 BUCK debited from Alice and credited to the `Notes` pool address.
 - A batch of N^* opaque `uint256` commitments (here $N^* = 16$: six live plus ten dummies) folded into the note Merkle tree, moving the root from `oldRoot` to the SNARK-attested `newRoot`, and the tree size from `nextLeafIndex` to `nextLeafIndex + N^*`.
 - One `Minted(issuer=Alice, totalFace=1200e18, startIndex=K, count=N*, newRoot)` event; no per-leaf `Appended` events – the commitments themselves live in the transaction calldata.
 - One `Transfer(Alice, Notes, 1200)` ERC-20 event.
 - One Groth16 proof (256 bytes, shape-independent of N) in calldata, accepted by the on-chain per- N verifier.
- Six off-chain artifacts, one per *live* recipient, each containing the *opening* of exactly one commitment plus the mint-transaction reference. The ten dummy openings are discarded or retained privately by Alice as anonymity- set filler.

The key invariant: looking at the chain alone, Mallory sees that Alice escrowed 1,200 BUCK and folded N^* hashes into the tree. She cannot tell which hash hides which value, who any recipient is, or even whether two of the recipients are the same person – or which entries are live notes vs.

zero-value padding. Looking at his off-chain artifact, Bob sees his 1,000 BUCK is real, was minted by Alice in `T_mint`, and that the *live* siblings among the other fifteen have hidden values summing to exactly 200 BUCK (the rest are $v = 0$ padding he cannot distinguish from live notes by chain inspection alone) – but he does not learn what those values are or who holds them.

3 What’s Inside a Note (Conceptually)

A "note" is just a five-tuple known by Alice and the recipient:

```
opening_i = ( flavor_i, v_i, rho_i, idHash_i, predicate_i )
cm_i      = Poseidon5( flavor_i, v_i, rho_i, idHash_i, predicate_i )
```

- **flavor_i** – which note variant (A1 identified cheque, A2 private cheque, B1 bearer cheque); see Notes for the menu.
- **v_i** – the face value in BUCK base units (18-decimal scaled integer).
- **rho_i** – a fresh, uniformly random nonce. This is the source of unlinkability: swapping **rho_i** changes **cm_i** to a uniformly fresh hash even if every other field is identical.
- **idHash_i** – a Poseidon-hashed handle on the recipient’s identity (e.g. a Poseidon-hash of the recipient’s registered ElGamal credential, or zero for a pure bearer note).
- **predicate_i** – a domain-separating tag for any optional spending condition (escrow, time-lock, etc.). Zero in the simplest case.

The Poseidon-5 hash gives the commitment two cryptographic properties that make the whole flow work:

- **Hiding.** Without **rho_i**, the commitment leaks no information about the other fields. Mallory sees **cm_i** and gains nothing about **v_i**, **idHash_i**, or **flavor_i**.
- **Binding.** No prover, hostile or honest, can find two distinct openings hashing to the same **cm_i** (collision-resistance of Poseidon on BN254, modelled as a random oracle in *alberta-buck-proofs.org* Part IV).

The on-chain contract stores only **cm_i**. Openings live exclusively off-chain.

4 The Mint, Step by Step

4.1 Step 0: Pre-flight (Off-Chain, In Alice’s Wallet)

Alice’s wallet has done all the slow work before any transaction is broadcast:

- She owns 1,200 BUCK on-chain.
- She has decided on the denomination split: [1000, 100, 25, 25, 25, 25]. $N=6$. The deployed mint pipeline pins a small set of batch sizes – recommended {16, 128, 1024} – one circom circuit and one verifier per pin. Alice’s wallet rounds her requested split up to the smallest pin $N^* > 6$, here $N^* = 16$, and pads with $16 - 6 = 10$ "blinded zero" openings whose $v_i = 0$. The padding leaves cost the same SNARK constraints as live notes but

contribute zero to `totalFace` and are spendable for zero BUCK; in effect they are anonymity-set filler whose openings Alice may discard. The narrative below tracks the six live notes and ignores the ten dummies (they show up in the on-chain commitments[] vector alongside the live ones, indistinguishable to Mallory).

- She has chosen an `idHash_i` for each recipient (typically a Poseidon hash of the recipient's IdentityRegistry-bound ElGamal credential, derived once at out-of-band introduction time).
- Her CSPRNG produces a fresh `rho_i` nonce per leaf – live or padded.

She now has 16 opening tuples in memory: 6 live, 10 dummies.

4.2 Step 1: Build Commitments

For `i = 0..15` (six live, ten dummies) her wallet computes

```
cm_i = Poseidon5( flavor_i, v_i, rho_i, idHash_i, predicate_i )
```

This is pure off-chain Poseidon arithmetic in `alberta_buck.wallet.notes`. Sixteen `uint256` field elements drop out. Their order in the eventual on-chain array is fixed by Alice's local list – the contract preserves whatever order she submits, and the SNARK assumes leaves land in the tree at indices `[nextLeafIndex, nextLeafIndex+1, ..., nextLeafIndex+N-1]` in submission order.

Alice's wallet must also keep a *local mirror of the rolling tree state*. The Tornado-style Merkle tree the contract maintains is parameterised by the standard `filledSubtrees[d]` for `d = 0..TREE_DEPTH-1` – the rightmost partially-built node at each level. The wallet snapshots this state from the chain (or replays it from the mint event log) immediately before assembling the proof, then derives, for each of the 16 leaves *in submission order*, the depth-20 sibling sequence the in-circuit insertion will consume. That derivation is the only delicate piece of new wallet code; everything else is unchanged from Phase-7.

4.3 Step 2: Generate the Batch-Mint Groth16 Proof

Alice's wallet runs `scripts/snark/prove_mint_batch.js` (the Phase-7-bis successor to Phase-7's `prove_mint.js`). The `mint_batch.circom` circuit at her chosen `pin N* = 16` takes a substantially richer witness than the Phase-7 mint did, because the Merkle-tree update has moved *into* the SNARK.

Public inputs (5 entries, N-independent):

```
oldRoot      # the live note root she read from chain
newRoot      # the root after folding her N leaves
nextLeafIndex # the live tree size she read from chain
totalFace    # 1200e18 -- range-bounded to 128 bits
cmBatchHash  # H(cm_0, cm_1, ..., cm_{N-1}) -- ties calldata to the SNARK
```

The hash `H(...)` is keccak256 (cheap on-chain; the contract recomputes it from calldata and asserts equality with the SNARK's bound value), or Poseidon if a future deployment chooses to keep the hash inside the field for in-circuit reuse.

Private witness (per-leaf and per-level):

```

For i in 0..N-1:
    flavor[i], v[i], rho[i], idHash[i], predicate[i]    # the opening
    pathSiblings[i][d] for d in 0..TREE_DEPTH-1        # 20 siblings per leaf

```

The `pathSiblings` for leaf `i` are derived by the wallet from its local `filledSubtrees` mirror plus the previously-folded leaves of *this same batch* (leaves `0..i-1` sit at known positions and contribute their Poseidon hashes to `i`'s path on the right). The wallet does this work; the chain consumes only `oldRoot`, `newRoot`, and the calldata `cms[]`.

In-circuit constraints:

1. **Per-leaf opening.** For each `i`, `cm[i] === Poseidon5(flavor[i], v[i], rho[i], idHash[i], predicate[i])`. `N` Poseidon-5 evaluations, `N` equality constraints.
2. **Range bounds.** Each `v[i]` and `totalFace` passes through `Num2Bits(128)`.
3. **Sum conservation.** `sum(v[0..N-1]) === totalFace`.
4. **Rolling Merkle insertion.** The circuit threads `oldRoot` and `nextLeafIndex` through `N` successive single-leaf insertions, mirroring the on-chain Phase-7 `Notes._insert` logic but in-circuit: at each step it folds `cm[i]` up the tree using `pathSiblings[i][*]` and `Switcher(...)` per level, advances the rolling state, and finally asserts the resulting root equals `newRoot`. `N x TREE_DEPTH` Poseidon-2 hashes ($\sim 4.3K$ constraints per leaf).
5. **Calldata binding.** `cmBatchHash == H(cm[0], cm[1], ..., cm[N-1])`.
6. (Optional belt-and-suspenders.) `cm[i] != 0` and `cm[i] != ZERO_VALUE` per leaf, blocking pathological collisions with empty-tree zeros.

Constraint counts work out to roughly **6K R1CS per leaf** ($\sim 1.7K$ opening + $\sim 4.3K$ insertion); at `N=16` that is $\sim 96K$ R1CS, a 2^{17} PTAU, and seconds of prover time on a desktop. At `N=128` it is $\sim 770K$ R1CS and minutes; at `N=1024` it is $\sim 6M$ R1CS and hours plus a sizeable memory footprint. The `scripts/snark/setup.sh` ceremony emits one verifier per pin.

The witness is processed by `snarkjs`'s Groth16 prover. Tens of seconds to minutes later the wallet has a fixed-size proof object (`pi_A: G1`, `pi_B: G2`, `pi_C: G1`), encoded as ABI-static (`uint256[2]`, `uint256[2][2]`, `uint256[2]`) – exactly 256 bytes regardless of `N`.

Observation. The Groth16 proof reveals *nothing* about the witness. Soundness guarantees that no `proofBytes` can satisfy the verification equation unless a witness exists; zero-knowledge guarantees that the verification equation reveals nothing about which witness was used. Mallory, even with the proof in hand, cannot back out `v[i]`, `rho[i]`, `idHash[i]`, `flavor[i]`, `predicate[i]`, or the per-leaf Merkle paths – only `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, and `cmBatchHash` leak.

4.4 Step 3: Submit Notes.mint On Chain

Alice now broadcasts a single transaction `T_mint` calling

```

notes.mint(
    proofBytes,
    oldRoot, newRoot, nextLeafIndex,
    1200e18,                // totalFace
    [cm_0, cm_1, ..., cm_15] // calldata, cms.length === N* (here 16)
);

```

The honest EVM walks `Notes.mint` (Phase-7-bis surface):

```
require(cms.length > 0, "Notes: empty mint");

// Stale-state guards: the prover must have read the live tree state.
require(oldRoot == roots[currentRootIndex], "Notes: stale oldRoot");
require(nextLeafIndex == self.nextLeafIndex, "Notes: stale leafIndex");

// SNARK verify against the dispatched per-N adapter.
bytes32 cmBatchHash = keccak256(abi.encodePacked(cms));
IMintVerifier mintVerifier = mintVerifiers[cms.length];
require(address(mintVerifier) != address(0), "Notes: unknown N");
require(mintVerifier.verifyMint(
    proofBytes, oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash
), "Notes: bad mint proof");

// Pull BUCK only after the proof is accepted.
require(buck.transferFrom(msg.sender, address(this), totalFace),
    "Notes: transfer failed");

// Adopt the SNARK-attested new root, advance the rolling state.
currentRootIndex = (currentRootIndex + 1) % ROOT_HISTORY_SIZE;
roots[currentRootIndex] = newRoot;
self.nextLeafIndex += cms.length;
noteFaceSum += totalFace;

// Single batch event; per-leaf Appended is gone. cms[] live in calldata.
emit Minted(msg.sender, totalFace, /*startIndex=*/ nextLeafIndex,
    /*count=*/ uint16(cms.length), newRoot);
```

Five things happen in strict order, and **all of them are atomic** (one transaction, one EVM state transition):

- a. **Stale-state guards.** The contract asserts that `oldRoot` equals the current ring-buffer head and `nextLeafIndex` equals the live tree size. Either fails if another mint landed since Alice's wallet snapshotted state (more on this under **Prover Contention and the Rollup Race** below).
- b. **Calldata binding.** The contract recomputes `cmBatchHash = keccak(abi.encodePacked(cms))` over the calldata it just received. This hash is one of the Groth16 public inputs, so any tampering with `cms[]` en route from the prover to the chain mismatches the SNARK and the verify call rejects.
- c. **SNARK verify.** The `MintVerifierAdapter` dispatches by `cms.length` to the per-N verifier (one circuit per pin, one verifier contract per pin), unpacks the 256-byte `proofBytes` into Groth16 form, builds the 5-entry public-signal vector `[oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash]`, and invokes the appropriate `MintGroth16Verifier`. If the verifier returns false, the entire transaction reverts – nothing is written, no BUCK moves.
- d. **BUCK escrow.** `buck.transferFrom` debits `totalFace` from Alice and credits the `Notes` pool. Allowance-driven pull, gated by Alice's prior `approve(notes, 1200e18, ...)`. Because the pool is registered *system-public* in the IdentityRegistry, the bilateral identity check on the transfer leg succeeds without a Chaum-Pedersen receipt.
- e. **Root advancement.** The contract writes `newRoot` into the next ring-buffer slot, advances `currentRootIndex` and `nextLeafIndex`, accumulates `noteFaceSum`,

and emits a single `Minted(issuer, totalFace, startIndex, count, newRoot)` event. The new root immediately becomes the live root that the next minter must read.

If any of (a)-(e) fails – stale root, stale leafIndex, bad proof, insufficient BUCK, missing approval – the entire transaction reverts and the chain state is indistinguishable from no submission at all. **Alice gets one shot per transaction**; she cannot retry a partial state.

Note on duplicate commitments. Phase-7 carried an explicit `commitmentExists[cm]` mapping that reverted on duplicate insertion. The batch-mint contract drops it – not because duplicates are harmless, but because they are now *economically self-punishing*: if Alice submits the same opening twice (whether across two batches or twice in one), she pays `totalFace` on each occurrence, she leaves land at distinct tree indices, but the spend nullifier `Poseidon3(rho, idHash, 4242)` is *deterministic in the opening*. After the first spend nullifies `cm`, the second copy is unspendable forever. Net result: Alice burns BUCK she could have kept. The chain’s job is value conservation, not saving Alice from a buggy `rho` RNG. See **Why Outside Observers Cannot Link** below and the per-attack-vector audit in alberta-buck-notes-rollup-mint.org.

4.5 Step 4: Off-Chain Note Delivery

After `T_mint` confirms, Alice transmits each live opening to its intended recipient by any out-of-band channel: an encrypted message, a printed QR code, a USB stick. The payload to recipient `i` is:

```
note_artifact_i = {
  txHash:      T_mint,                # Ethereum transaction ID
  leafIndex:   startIndex + i,        # absolute position in the commitment vector
  cm:          cm_i,                  # for cross-checking against calldata
  opening:     ( flavor_i, v_i, rho_i, idHash_i, predicate_i ),
  meta:        { issuer: Alice's IdentityRegistry handle, batchSize: N*, ... },
}
```

The chain itself never sees `opening`. `cm_i` is already folded into `newRoot`; the artifact carries it explicitly only as a convenience so the recipient does not have to recompute it before locating it in the batch’s calldata `cms[]`.

The ten dummy openings Alice generated to pad to the pinned `N=16` circuit receive no artifact; they are discarded (or, at Alice’s option, retained privately as unspendable anonymity-set filler). From the recipient’s perspective the presence of dummies is invisible: Bob’s artifact only names his own leaf, and the batch total `totalFace = 1,200 BUCK` is exactly the sum of the live `v_i`.

4.6 Step 5: Recipient Verification (Offline, Independent)

Bob receives his artifact for the 1,000-BUCK note. Without trusting Alice and without any help from an indexer, he runs:

1. **Recompute and check.** Bob computes `cm_check = Poseidon5(flavor, 1000e18, rho, idHash, predicate)` and asserts `cm_check == cm`. If this fails, Alice lied about the opening; Bob discards the artifact and demands a real one. (Binding of Poseidon makes substitution impossible.)
2. **Look up the mint transaction.** Bob reads `T_mint` on-chain, parses the call data into `(proofBytes, oldRoot, newRoot, nextLeafIndex, totalFace, cms[])` and reads `msg.sender` from the transaction envelope.

3. **Confirm msg.sender is Alice.** Cross-check `msg.sender` against the IdentityRegistry: it must resolve to Alice’s PS-credentialed identity. If Bob wanted an A1 identified cheque, the issuer field of his opening must match.
4. **Confirm calldata membership.** Assert `cms[leafIndex - nextLeafIndex] = cm` (where the artifact’s `leafIndex` is the absolute position). The chain’s acceptance of `T_mint` confirms that every entry in `cms[]` was folded into `newRoot` and that `newRoot` was added to the root ring buffer. Bob therefore has chain-anchored evidence that `cm` is officially a commitment in the pool.
5. **Confirm batch totals.** Read `totalFace` from the call data. The chain’s acceptance of `T_mint` is itself a proof that the Groth16 verifier accepted – which means `sum(v[0..N-1]) == totalFace` holds in the witness. Bob does not need to re-verify the proof; the EVM already did. But if he wants belt-and-suspenders, he can re-run `MintGroth16Verifier.verify(proofBytes, [oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash])` locally against the same on-chain bytecode – and `cmBatchHash` he recomputes from the calldata `cms[]` using the same `keccak256(abi.encodePacked(cms))` the contract uses.

After (1)-(5) Bob knows:

- His note encodes exactly 1,000 BUCK. (Step 1.)
- It was minted in `T_mint` by Alice. (Steps 2, 3.)
- It is one of N^* (here 16) commitments folded into the tree by that mint, the others being `cms[] \ cm`. (Step 4.)
- The *live* values among the other siblings sum to exactly $1200 - 1000 = 200$ BUCK (the residual of `totalFace` once Bob subtracts his own `v`); the batch may also contain dummy openings with `v = 0`. (Step 5 + the per-note opening relation: he knows *his* value, the chain attests to the *sum*.)
- He learns nothing else about the other notes – not their individual values, not their recipients, not their flavors, not which are live and which are padding. Each `cm_j` is hiding under a fresh `rho_j` he does not have.

This is the unlinkability promise made concrete: each recipient can pin down their own slice and the batch total, without learning anything about siblings.

5 Why Alice Cannot Cheat (Even With a Hostile EVM and Prover)

Alice runs whatever software she wants on her laptop. She can fork the snarkjs witness generator, patch the circom compiler, run Anvil with a malicious `MintGroth16Verifier` that always accepts. None of it matters, because the truth on the public chain is determined by what *the honest validators* run. Three pillars prop up the soundness story:

5.1 Pillar 1: The On-Chain Verifier Is Fixed Code

`MintGroth16Verifier.sol` (one per pinned N) was emitted from the Groth16 verification key produced by the trusted setup (Phase 9 for production; dev-only entropy in `scripts/snark/setup.sh`

today). Its bytecode is deployed once and immutable. Validators run whatever bytecode sits at that address; Alice cannot tamper with it without consensus-breaking the chain.

Knowledge soundness of Groth16 means: for every `proofBytes` the verifier accepts on public inputs `[oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash]`, there exists a witness `(flavor[i], v[i], rho[i], idHash[i], predicate[i], pathSiblings[i][*])_i` such that:

- `cm[i] = Poseidon5(flavor[i], v[i], rho[i], idHash[i], predicate[i])` for all `i`
- each `v[i]` and `totalFace` fit in 128 bits and `sum(v[i]) = totalFace`
- folding `cm[0..N-1]` into the tree rooted at `oldRoot`, starting at position `nextLeafIndex`, using `pathSiblings[i][*]` as the right-of-insert context, yields exactly `newRoot`
- `cmBatchHash = H(cm[0], ..., cm[N-1])`.

Alice is free to *find* such a witness any way she likes – she’s the prover. But she cannot find a witness for a *false* statement; that’s what soundness prohibits.

5.2 Pillar 2: BUCK Escrow Is Inside the Same Atomic Call

The proof check, the BUCK pull, and the root advancement all live in the same `Notes.mint` function. An EVM transaction is all-or-nothing: the only two outcomes are "the entire call’s state changes commit" or "the entire call’s state changes revert".

This means Alice cannot, for instance:

- Verify a proof, then refuse to pay – the verifier acceptance and the `transferFrom` live in the same call, gated by the same `require`.
- Pay 1,200 BUCK but then get away with advancing only part of the tree – `newRoot` is SNARK-attested as the deterministic fold of all of `cms[]` onto `oldRoot` at `nextLeafIndex`, and the contract writes it in one store.
- Replay a prior `proofBytes` – the SNARK is bound to `(oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash)` and the contract’s stale-state guards reject any submission whose `oldRoot` no longer matches the live root or whose `nextLeafIndex` no longer matches the live tree size.
- Submit the same opening twice *in one batch* to double-spend at spend time – the nullifier is deterministic in `(rho, idHash)`, so whichever duplicate lands first at spend will consume both. (See the duplicate-commitment note under Step 3.)

5.3 Pillar 3: Public Inputs Are Sealed Into the Proof

The Groth16 verifier consumes the public-signal vector `[oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash]`. The proof was generated against the specific values Alice committed to before submission. If Alice tries to submit the proof with a different `totalFace`, a different `newRoot`, a different `nextLeafIndex`, or a permuted `cms[]` whose keccak no longer matches `cmBatchHash`, the verifier rejects.

Note that `cms[]` itself is **not** a public input to the SNARK – the proof binds only to its hash `cmBatchHash`. The contract is the component that recomputes that hash from calldata on every call, so any mismatch between the proved hash and the recomputed hash reverts at verify time.

The refreshed `test/MintVerifier.t.sol` should cover at least these tamper paths, each expected to revert with `Notes: bad mint proof` or `Notes: stale oldRoot` or `Notes: stale leafIndex` or an ABI decode error:

- `totalFace + 1`
- `newRoot` flipped by one bit
- `oldRoot` spoofed to a stale value
- `nextLeafIndex` spoofed to a fictional size
- `cms[0] ^ 1` (commitment tampered in calldata after proving)
- `cms[]` length differs from the dispatched verifier’s pinned `N`
- malformed 256-byte `proofBytes`

Soundness is not a proof of *absence* of attack, but every documented attack surface is closed by a test that triggers a verifier reject.

5.4 Putting the Pillars Together: The Adversarial Walk

Suppose hostile-Alice tries to mint two **contradictory** batches that both claim to be the canonical breakdown of her `T_mint` transfer. She would need:

- Two different `proofBytes` and/or `cms[]`, both passing the on-chain verifier.

That requires either:

- Two valid Groth16 witnesses for the same public inputs (allowed – but they yield the same `cms`, so they are not *contradictory*), or
- Two valid Groth16 witnesses with *different* public inputs. Allowed too, but each one demands its own `transferFrom` for its own `totalFace` – **that** is the duplicate-spend problem ERC-20 solves. Alice cannot transfer the same 1,200 BUCK twice without holding 2,400 BUCK to start with.

So a single 1,200-BUCK escrow yields exactly one accepted batch. The only freedom Alice has is *which* batch she lays down – which denominations and which recipients – and that freedom is hers to keep, by design.

What about hostile-Alice trying to mint *more* commitment value than she escrowed? She would need a witness whose `sum(v[i])` equals the integer `totalFace` *that the EVM sees*, but whose individual `v[i]` values *actually* sum to more. In `F_r` with $r \sim 2^{254}$ and no range check on `v[i]`, the constraint `sum(v[i]) == totalFace` would be satisfiable by witnesses whose integer sum differs from `totalFace` by a multiple of `r`. `mint_batch.circom` closes this with a `Num2Bits(128)` range constraint on each `v[i]` and on `totalFace`, mirrored by `spend.circom`’s check on `face` and the witness `v`. A 128-bit cap is comfortably above any plausible BUCK denomination and well below $r/2$, so an integer overshoot can never wrap.

What about hostile-Alice trying to mint a commitment that is *already* in the tree (to collide on a nullifier she does not control, or to attempt a hash-collision attack)? She cannot collide on a nullifier without knowing somebody else’s opening – the nullifier is a Poseidon-PRF output, so

forging a collision is as hard as inverting Poseidon. She *can* re-mint her own opening (the contract no longer blocks duplicates), but doing so forces her to pay `totalFace` a second time and buys her nothing: the nullifier is deterministic in the opening, so whichever of her two copies she spends first permanently nullifies both. This is the value-conservation property in action – the chain does not need a `commitmentExists` check because the nullifier mechanism already collapses duplicate openings to one spend, and Alice bears the cost of any duplication.

Full per-attack-vector analysis (twelve attack vectors A1-A12) is in the pivot design memo alberta-buck-notes-rollup-mint.org. Every hostile-Alice guarantee from the Phase-7 per-leaf insertion design survives under the batch-mint pivot; the only behavioural change is that duplicate-commitment rejection moves from an explicit on-chain check to economic self-enforcement via the deterministic nullifier.

6 Why Outside Observers Cannot Link

Mallory has full read access to the chain after `T_mint` confirms. What does she see?

`T_mint`:

```
from: Alice (a registered IdentityRegistry account)
to:    Notes (a system-public contract)
value: 0 (BUCK is ERC-20, not ETH)
data:  Notes.mint(proofBytes, oldRoot, newRoot, nextLeafIndex,
                  1200e18, [cm_0..cm_15])
```

events emitted (in order):

```
Buck.Transfer(Alice, Notes, 1200e18)
Notes.Minted(Alice, 1200e18, startIndex=K, count=16, newRoot)
```

The batch-mint contract emits one `Minted` event per batch rather than per-leaf `Appended` events; the commitments themselves live in calldata for off-chain provers and wallets to read. (Calldata is about an order of magnitude cheaper per byte than LOG data, which is part of why the N-independent cost bound holds.)

What can Mallory infer?

- That Alice (publicly) escrowed 1,200 BUCK. **Yes** – her address is `msg.sender`.
- That sixteen new commitments entered the pool at indices `[K..K+15]`. **Yes** – `Minted` announces `startIndex` and `count`, and `cms[]` is in calldata.
- *Which* commitment is worth *how much*. **No** – each is a fresh `Poseidon5` output hidden by an independent `rho_i`. Without a `rho_i`, distinguishing the 1,000-BUCK hash from the 25-BUCK hash (or from a 0-BUCK dummy) is computationally equivalent to inverting Poseidon on a uniform field element, modelled here as a random oracle.
- */Which* entries are live notes and which are dummies.* **No** – the padding leaves are constructed with the same Poseidon-5 shape and a fresh `rho`; nothing about them is chain-distinguishable from a live `cm` with a small `v`.
- *Who* holds each live note. **No** – `idHash_i` is also inside the hash.

- Whether two commitments share a recipient. **No** – two notes addressed to the same `idHash` but with distinct `rho_i` produce uniformly independent `cm_i`, so the chain shows nothing distinguishable from independent draws.

What about *linking back to a specific note artifact* that Bob later possesses? Mallory sees `cm_0..cm_5` published by Alice in `T_mint`. Bob's artifact references some `cm_i` and `T_mint`, but the artifact lives off-chain. Mallory cannot get the artifact unless Bob (or Alice) shows it to her, and the chain alone gives her no way to single out `cm_i` as "the one Bob holds" against the other five.

The remaining linkability surface is the issuer. `msg.sender` is publicly Alice, so every commitment in the batch carries a public **issuer** tag. This is intentional and desired: a recipient must be able to verify who minted his note. The privacy goal is unlinkability of recipients and amounts, not of issuers.

6.1 Spend-Time Unlinkability

The shipped spend circuit (Phase 7) lets Bob deposit his note without revealing *which* `cm` in the on-chain set he opened. The spend SNARK proves "I know an opening to *some* commitment in the Merkle tree under the public `noteRoot` that hashes to this nullifier", with `cm` as a private witness rather than a public input. Unlinkability of *spend events* to *mint events* is now a chain-observable property: Mallory sees a fresh nullifier, a public face, and a recipient – nothing that points back to a specific commitment in the tree. The full walkthrough of the redemption flow is in **The Redemption (Spend), Step by Step** below.

7 Atomicity, No Double-Mint, No Replay

Three properties together rule out duplicated or partial mints:

7.1 Atomicity

Already covered above: `Notes.mint` performs proof verification, BUCK pull, and commitment appends in one EVM call. Either all happen or none.

7.2 No Double-Mint From One Transfer

The transfer and the commitments are bound in the same call: there is no "pre-deposit, then mint" or "mint, then deposit later" flow that an attacker could desynchronize. Each `Notes.mint` invocation does its own `transferFrom`; each `transferFrom` debits Alice's BUCK balance once. Two minting transactions require two `transferFrom` debits, which require Alice to actually hold $2 * \text{totalFace}$ BUCK.

7.3 No Replay

A literal replay – broadcasting the exact same `T_mint` calldata in a new transaction – fails for three independent reasons:

a. **Stale-state guards.** The contract checks `oldRoot = roots[currentRootIndex]=` and `nextLeafIndex = self.nextLeafIndex=`. The first mint has already advanced both, so the replay's `oldRoot` and `nextLeafIndex` public inputs no longer match the live chain state. The contract reverts before the SNARK is even called, with `Notes: stale oldRoot` or `Notes: stale leafIndex`. b. **SNARK binding.** Even if a future minter coincidentally re-created the same `oldRoot` and `nextLeafIndex`

state (which is prohibited by the monotonic advancement of `nextLeafIndex`), the SNARK was bound to the specific (`cmBatchHash`, `newRoot`) pair of the prior batch. A replay would fold the same `cms[]` a second time and produce a different `newRoot` than the SNARK attested to, breaking the rolling-tree relation. c. **Allowance and balance.** Even setting the above aside, the second mint would re-debit Alice’s BUCK and re-consume her allowance to the `Notes` pool – economic-level double-escrow protection on top of the structural guards.

Replay of the same opening across batches (the replacement of the Phase-7 `commitmentExists` check) is structurally allowed but economically self-punishing: Alice escrows `totalFace` again, the duplicate commitment lands at a fresh leaf index, and the deterministic nullifier `Poseidon3(rho, idHash, 4242)` means the first spend nullifies every copy. Alice pays twice for a spend-once outcome.

7.4 Cross-Chain or Cross-Deployment Replay

The `MintGroth16Verifier` verification key is fixed by the trusted-setup ceremony for that deployment. A proof generated against the dev-key in `scripts/snark/setup.sh` will not verify against a production-key contract on mainnet, and vice versa. Public inputs are also sealed into the proof: a proof valid on Sepolia for `totalFace=1200` cannot be reused on mainnet because the on-chain verifier address (and key) differ.

The only cross-context vulnerability would be a *chain-id collision* on identical verification keys, which the production setup ceremony precludes by construction.

8 Prover Contention and the Rollup Race

The batch-mint pivot introduces one genuinely new operational concern that did not exist in the Phase-7 per-leaf insertion design: a mint transaction is now a *rollup-style* transaction. Alice’s proof is bound to the specific `oldRoot` and `nextLeafIndex` she snapshotted when she began proving; if any other mint lands between her snapshot and her submission, those values go stale and her transaction reverts at one of the stale-state guards.

This is not a *safety* concern – Alice loses no BUCK (the contract reverts before `transferFrom` runs) and produces no bad on-chain state. It is a *liveness and efficiency* concern: Alice wasted the prover time spent constructing a proof that is no longer accepted, and has to re-prove against the fresh state.

8.1 Who Wins When Two Minters Race?

When Alice and Bob both submit `T_mint_A` and `T_mint_B` targeting the same `oldRoot`:

- One of the two (by block ordering) lands first and updates the root.
- The other transaction executes, discovers the stale-state mismatch, and reverts at the `require(oldRoot == roots[currentRootIndex])` line before any BUCK moves.

Transaction fees are paid by the loser for the reverted transaction (gas for the revert path, not for the full mint). The loser then re-proves against the new `oldRoot` and `nextLeafIndex` and resubmits. The new proof folds *their* commitments onto the root that *now* includes the winner’s commitments – no commitment has been lost; the loser’s tree-position is just different from the one they originally predicted.

8.2 Mitigation Layers

Three mitigation layers, in ascending complexity:

1. **Optimistic retry (wallet-level).** On revert, the wallet re-fetches (`currentRoot`, `nextLeafIndex`) from the chain, rederives the sibling paths for the new insertion positions, re-runs the prover, and resubmits. For small N this is fast enough to not matter. This is the recommended initial deployment strategy.
2. **Retry budget and back-off (wallet-level).** For large N where prover time is nontrivial (minutes to hours at $N=1024$), the wallet should expose a "chance of contention" estimate to the user and offer to batch at the smallest pinned N that fits the job, or to enqueue the mint on a private relay channel during low-traffic windows.
3. **Sequencer / proof aggregation (future).** If real-world contention warrants it, an external sequencer can batch multiple users' mints into a single proof per slot, preserving the N -independent on-chain cost while amortising prover time. Sequencer capture is a separate trust axis and is deferred; the shipped protocol is sequencer-free.

8.3 What Does Not Regress

- **Atomicity.** A reverted mint leaves *no* chain-visible state change; it is strictly indistinguishable from no submission.
- **Value conservation.** No reverted transaction debits BUCK, so there is no way for a loser to pay the pool without appending commitments.
- **Unlinkability.** A re-submitted mint on a fresh root still produces the same (anonymity-preserving) calldata and event shape; Mallory cannot tell a retry from a first attempt.
- **Spend-time invariants.** The spend circuit takes the current `noteRoot` (or any recent-window root); a late-arriving mint does not invalidate any previously-proved spend.

The rollup race is the price of replacing $\sim 800K$ gas/leaf with ~ 500 gas/leaf, and for practical block-rate contention it is negligible.

9 Receipt Construction From the Transaction ID Alone

A practical question: months later, Bob wants to prove to Carol that his note was legitimately issued. He needs no help from Alice and no live indexer:

1. Bob fetches `T_mint` from any Ethereum archive node.
2. He decodes the call data into (`proofBytes`, `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cms`) using the standard `Notes.mint` ABI.
3. He notes `msg.sender` and resolves it to Alice's identity via the `IdentityRegistry` on the same chain (a single `eth_call`).
4. He locates his `cm` in `cms[]` at `leafIndex - nextLeafIndex` and asserts equality.
5. He recomputes `Poseidon5(flavor, v, rho, idHash, predicate) == cm` from the opening.

6. (Optional.) He recomputes `cmBatchHash = keccak256(abi.encodePacked(cms))` and re-runs `MintGroth16Verifier.verify(proofBytes, [oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash])` against the per-N verifier keyed by `cms.length`, convincing himself the EVM did its job.

The "receipt" Carol receives is therefore `(T_mint, opening, idHash mapping)`. Its validity is independently checkable against the public chain. Nothing here requires ongoing cooperation from Alice – which is exactly the property a receipt should have.

10 The Redemption (Spend), Step by Step

This section walks the *inverse* of the mint. Bob holds the opening for the 1,000-BUCK note `cm_0`; he wants to redeem it for 1,000 BUCK credited to his on-chain account, and he wants no public observer to be able to tell *which* commitment in the pool he consumed. The shipped pieces are `circuits/spend.circom`, `SpendGroth16Verifier.sol`, `SpendVerifierAdapter.sol`, `Notes.spend()`, and the Tornado-style on-chain Merkle accumulator inside `Notes.sol`; everything below describes *code paths that are in the tree and exercised by test/SpendVerifier.t.sol*. The companion theorems live in Theorems 8-10 of `alberta-buck-proofs.org`.

Bob's note is of flavor *A1* (addressed, identified cheque), but the shipped spend circuit is *flavor-agnostic*: it enforces a B-shape (bearer) discharge in which knowledge of the Poseidon-5 opening plus the Poseidon-3 nullifier preimage is the spend authorization. In particular the in-circuit ElGamal decryption and Chaum-Pedersen equality that would distinguish A-flavor from B-flavor are deferred to a future `spend_a.circom`; until that ships, A-flavor notes are effectively bearer-spendable at the SNARK boundary, with the on-chain wallet conventions and the pool's identity-bound transfer surface still applying. The walk below uses Bob's A1 note as the worked example, with notes inline where A-flavor identity binding will tighten the picture once `spend_a.circom` lands.

Three properties of the flow are worth stating up front, because they shape every step:

- **The anonymity set is the full pool, not only same-denomination.** The spend proof takes `cm` as a *private* witness under a public `noteRoot` – the chain-observable anonymity set is every unspent commitment in the tree. The **face value is a public output**, because the BUCK contract must know how much to pay out; amount-level linkability narrows the effective set to commitments whose hidden `v` could equal the revealed **face**. Same-denomination issuance is therefore the strongest-privacy regime, but the underlying cryptographic hiding is over the whole tree, not over a denomination class.
- **The BUCK ERC-20 does not verify the SNARK itself.** The authorization chain is: `valid SNARK + accepted root + fresh nullifier` → `Notes.spend` proceeds → `Notes.spend` invokes `buck.transferCarrying(recipient, face)` from the pool. The proof is a capability token for entry into `Notes.spend`; the resulting ERC-20 movement is a routine pool-to-recipient call gated by that entry. Since the `Notes` pool is a system-public account in the IdentityRegistry, the pool's outbound transfer does not itself require a Chaum-Pedersen receipt – the recipient's own registration is what makes them acceptable as a transfer destination.
- **Public inputs are bound into the proof, including `chainId`.** `spend.circom` ghost-binds `recipient` and `chainId` (via `x*x` rows) so Groth16's `IC[]` commitment makes them non-malleable from the mempool. A front-runner who steals Bob's `proofBytes` and resubmits with a different `recipient` or on a different chain id is rejected by the verifier.

10.1 Step 0: Bob Holds a Verified Note

From the mint walk (**Step 5: Recipient Verification**) Bob already has:

```
note_artifact_0 = {
  txHash:    T_mint,
  index:     K + 0,
  cm:        cm_0,
  opening:   ( flavor_A1, 1000e18, rho_0, idHash_bob, predicate_0 ),
  meta:      { issuer: Alice, ... },
}
```

He has independently confirmed that `cm_0` is in `Notes.commitments[]` at index `K` and that `Poseidon5(opening)` reproduces it. The opening has been sitting in his wallet for weeks, months, or years – addressed A-notes are cryptographically safe for indefinite holding (`Notes`).

10.2 Step 1: Fetch the Commitment Tree and Build a Merkle Path

`Notes.sol` maintains a Tornado-style incremental Poseidon-T3 Merkle tree of `TREE_DEPTH = 20` (max ~1M leaves). Under Phase-7-bis the per-leaf insertion math lives inside the mint SNARK; the contract just records the SNARK-attested `newRoot` at each `mint()` call. The current root is exposed as `noteRoot()`. Two things happen:

- a. Bob (or, more practically, any light-node helper running off-chain) reconstructs the tree off-chain. The commitments themselves are available from each mint transaction’s calldata `cms[]`, keyed by the `Minted(issuer, totalFace, startIndex, count, newRoot)` event that indexes the batch. A replayer walks all `Minted` events in order, takes the `cms[]` slice out of each mint’s calldata, and appends them to its local tree using the same `ZERO_VALUE` and Poseidon-T3 wrapper the in-circuit insertion uses. Bob’s Merkle path for `cm_0` – a sequence of sibling hashes plus left/right indices up to the root – falls out of that rebuilt tree. The path is purely public information; everyone can reproduce it, and Bob does not have to trust any indexer because calldata is canonical.
- b. Bob reads the current `noteRoot()` via `eth_call`. He may instead pin any root in the recent-roots window: `Notes.sol` retains the last `ROOT_HISTORY_SIZE = 30` roots in a ring buffer (`roots[]`, advanced once per inserted leaf), and the on-chain `isAcceptedRoot()` scans that ring backward from `currentRootIndex`. This is the standard zk-rollup hygiene (Zcash, Tornado Cash) for tolerating prover latency: a spend assembled against a slightly-stale root remains acceptable so long as fewer than 30 mints have landed in the meantime.

10.3 Step 2: Compute the Nullifier

The shipped `spend.circom` derives the nullifier as a Poseidon-3 hash of the witness `rho`, the witness `idHash` bound at mint, and a constant tag:

$$nf = \text{Poseidon}_3(\rho_0, \text{idHash}_0, 4242).$$

The tag (4242) is a fixed nonzero field constant that domain-separates nullifier preimages from the Poseidon-5 commitment preimages, so a structural collision between a nullifier and a commitment is impossible. Three properties follow:

- **Deterministic.** The same `(rho, idHash)` always yields the same `nf`. So Bob can only spend `cm_0` once; a second attempt re-derives the same `nf` and is rejected by `nullifiers[nf] = true` on the first spend.

- **Unforgeable without the opening.** An attacker who sees `cm_0` and the tree cannot compute `nf` without `rho_0` and `idHash_0`, both of which live inside the opening (and `rho_0` is not derivable from the recipient’s identity alone).
- **A-flavor binding is not yet in the SNARK.** The bearer-shape circuit treats `idHash` as a witness scalar – it does not constrain it to the recipient’s registered identity. Once `spend_a.circom` ships, the in-circuit Chaum-Pedersen

– ElGamal decryption will pin `idHash` to Bob’s registered `M_rec`, making `nf`

re-issuance-safe under Identity rotation (Theorem 10.1). Until then, `idHash` unlinkability for A-flavor notes rests on it being secret in the opening, not on a registry-bound CP equality.

The nullifier is the chain-observable anchor of the spend. Publishing it marks `cm_0` as consumed from the tree’s *semantic* point of view, even though the tree itself remains append-only and nothing is deleted.

10.4 Step 3: Generate the Spend Proof

Bob’s wallet runs `scripts/snark/prove_spend.js` (the spend-side analogue of `prove_mint_batch.js`, using the depth-20 PTAU produced by `scripts/snark/setup.sh`). `spend.circom`’s signal layout is exactly:

```
public noteRoot;           # an accepted root from the on-chain ring buffer
public nullifier;          # the nf from Step 2
public face;               # 1000e18 -- the BUCK to release (range-bounded to 128 bits)
public recipient;          # Bob’s on-chain address (the payout destination)
public chainId;            # replay-domain separation

private flavor, v, rho,
      idHash, predicate;    # the Poseidon-5 opening
private pathElements[depth]; # depth-20 sibling hashes
private pathIndices[depth];  # left/right bit per level
```

The shipped circuit enforces five relation groups:

1. **Range bounds and value match.** Both the public `face` and the witness `v` pass through `Num2Bits(128)`, then `face == v`. This closes the field-overflow gap: `v` cannot wrap around `r` to claim more than its in-range share.
2. **Opening binds.** `cm = Poseidon5(flavor, v, rho, idHash, predicate)`.
3. **Merkle membership.** `MerkleProof(20)` walks `(cm, pathElements, pathIndices)` up the tree using `Switcher + Poseidon(2)` per level and asserts the climb reaches `noteRoot`.
4. **Nullifier derivation.** `nullifier == Poseidon3(rho, idHash, 4242)`.
5. **Public-input ghost binding.** `recipient` and `chainId` each appear in a trivial `x*x` row. This is the minimal anchor needed for circom not to optimise out unreferenced public signals; once anchored, Groth16’s `IC[]` commitment makes them non-malleable from the mempool’s perspective.

Critically, `cm` is *not* a public input; it is recomputed inside the proof from the witness opening. The proof convinces the verifier that *some* `cm` in the tree satisfies (1)-(4); it does not reveal *which*.

The two A-flavor relations from earlier drafts – in-circuit ElGamal decryption of the note’s `E_note` payload, and a Chaum-Pedersen equality against the depositor’s registered credential – are deferred to a follow-up `spend_a.circom`. Until that ships, the bearer/B-spend semantics apply uniformly: knowledge of the Poseidon-5 opening *is* the spend authorization. For genuine A-flavor protection, wallet software relies on `idHash` secrecy in the off-chain artifact and on the pool’s identity-bound transfer surface (the `transferCarrying` recipient check), not on in-SNARK CP equality.

Bob’s wallet serializes the Groth16 proof into the standard `(uint256[2], uint256[2][2], uint256[2])` ABI-static tuple (256 bytes).

10.5 Step 4: Submit `Notes.spend` On Chain

Bob now broadcasts the spend transaction `T_spend`:

```
notes.spend(proofBytes, noteRoot, nullifier, 1000e18, bob);
```

The honest EVM walks `Notes.spend` (the function exactly as it ships):

```
require(recipient != address(0), "Notes: zero recipient");
require(face > 0, "Notes: zero face");
require(!_isAcceptedRoot(root), "Notes: unknown root");
require(!nullifiers[nullifier], "Notes: already spent");
require(spendVerifier.verifySpend(
    proof, root, nullifier, face, recipient, block.chainid
), "Notes: bad spend proof");

nullifiers[nullifier] = true; // mark spent before paying out
noteFaceSum -= face;

require(IBuckCarrying(address(buck))
    .transferCarrying(recipient, face),
    "Notes: transfer failed");
emit Spent(nullifier, face, recipient);
```

Four invariants are enforced in order:

- a. `root` appears in the recent-roots window. A root that has rotated out is stale – Bob must rebuild the proof against a fresher root.
- b. `nullifier` has never been published before (Theorem 10.5 in the proofs says this single-bit check is sufficient for double-spend prevention).
- c. The SNARK is valid against `SpendGroth16Verifier` via the `SpendVerifierAdapter`, which packs `[root, nullifier, face, recipient, chainid]` into the public-signal vector and unpacks `proofBytes` to Groth16 form. By soundness, a witness satisfying (1)-(4) above exists.
- d. Only after (a)-(c) pass does the contract mark the nullifier and invoke `buck.transferCarrying(bob, 1000e18)` from its own pool balance (see Step 5). Because the `Notes` pool is registered `isPublic` in the `IdentityRegistry`, the outbound transfer’s bilateral identity check degenerates to "recipient is a registered or system-public account" and skips the Chaum-Pedersen leg.

If any of (a)-(d) reverts, the whole transaction reverts: no BUCK moves, the nullifier bit is never set, the `noteFaceSum` is unchanged. **Bob gets one attempt per root window**; a failed spend

can be retried with a refreshed proof, but never in a way that consumes a nullifier without paying out the BUCK.

10.6 Step 5: BUCK ERC-20 Pays Out (transferCarrying)

The call inside `Notes.spend` is `buck.transferCarrying(bob, 1000e18)`, a small extension of the standard BUCK transfer that the `Notes` pool uses on its single egress. Nothing else in `Buck.sol` is special-cased for the pool: it is just a registered account making an outbound transfer of 1,000 BUCK to a registered recipient. The result on-chain:

- `Buck.balanceOf(Notes)` decreases by exactly 1,000 BUCK – nominal balance only.
- `Buck.balanceOf(Bob)` increases by exactly 1,000 BUCK.
- `Buck.Transfer(Notes, Bob, 1000e18)` is emitted.
- `Buck.TransferCarrying(Notes, Bob, 1000e18)` is emitted as the carry signal.
- `Notes.Spent(nullifier, 1000e18, Bob)` is emitted.
- `Notes.nullifiers[nullifier]` becomes `true`.

What "carrying" means in the ledger. BUCK demurrage uses the standard cumulative-index accounting: a global `cumIndex` advances at `BASE_RATE_PER_SEC` and each account stores its `_indexAtLastTouch`. A standard transfer settles the sender's accrued fee at transfer time (deducting it as a one-shot burn) and merges the recipient's index toward the global `cumIndex` weighted by the incoming amount. `transferCarrying` differs in two surgical ways:

1. **Sender's index is unchanged.** The sender's remaining balance keeps its pre-transfer age; no fee is burned at the sender on this transfer.
2. **Recipient absorbs the sender's age.** The recipient's `_indexAtLastTouch` is updated by a balance-weighted average toward the *sender's* pre-transfer index (`idxFrom`), not toward the global `cumIndex`:

```
_indexAtLastTouch[to] =  
    (br * _indexAtLastTouch[to] + amount * idxFrom) / (br + amount);
```

System-wide BUCK-age is preserved across the call (no fee is burned, no fresh-BUCK dilution at the recipient); the demurrage that the pool would have burned on a standard outgoing transfer instead rides into Bob's account as a forward index shift. Bob will pay it the next time he makes a standard outgoing transfer.

Why this is the right rule for the pool. Per-note age is opaque – the pool holds aggregate BUCK behind opaque commitments and cannot tell Bob's note's age from any other note's. `transferCarrying` collapses the calculation to the pool's average index at spend time: every spender of the same face absorbs the same forward index shift, keeping the spend-time anonymity set intact. The trade-off is that Bob inherits the pool's average age regardless of how long *his specific* note sat in the tree. Anyone whose note rested longer than the pool average comes out ahead; this is the cost of anonymity, and the upside for long-term holders.

The full discussion (one new BUCK method, one new event, no per-account mode flag) is in **Demurrage-Carrying Transfers** below.

10.7 What the Observer Sees at Spend (Mallory Returns)

After `T_spend` confirms, Mallory can read:

`T_spend`:

```
from: Bob (a registered IdentityRegistry account)
to:    Notes (system-public)
data:  Notes.spend(proofBytes, noteRoot, nullifier, 1000e18, Bob)
```

events:

```
Buck.Transfer(Notes, Bob, 1000e18)
Notes.Spent(nullifier, 1000e18, Bob)
```

What she infers:

- **That a note was spent.** Yes – the nullifier is fresh and the pool balance dropped by 1,000 BUCK.
- **That the spend was for 1,000 BUCK to Bob.** Yes – `face` and `recipient` are public inputs (they must be, to tell the ERC-20 whom to pay).
- *Which commitment in the tree was consumed?* **No** – `cm` is a private witness to the SNARK. Mallory’s anonymity set is every unspent commitment of flavor `A` in the pool whose hidden `v` could equal 1,000 BUCK. At deployment scale this is potentially the entire pool minus prior nullified entries; in a same-denomination regime (many 1,000-BUCK notes in circulation) it is exactly the 1,000-BUCK subset.
- *Which mint did the spent note come from?* **No** – there is no cross-reference to `T_mint` in `T_spend`. The only link back to any specific mint would be the nullifier, but nullifiers are Poseidon-PRF outputs over secret material; inverting one to recover (`flavor`, `M_rec`, `rho`) is exactly the pre-image resistance we assume.
- *Who held the note before Bob?* **No**. If Alice gave the note to Carol who handed it to Dave who sold it to Bob, none of those transfers touched the chain. The chain sees only "at mint Alice deposited, later Bob withdrew" – the intermediate holders are invisible.

The residual linkability surfaces are (i) **face** (the revealed amount) and (ii) **recipient** (the on-chain payout address). Both are unavoidable at the SNARK/ERC-20 boundary – the contract must know where and how much to pay. The community- standardized mitigation is fixed-denomination notes (so revealing **face** does not subdivide the anonymity set) and batched, relay-mediated payouts (so revealing **recipient** need not reveal the *true* beneficiary). Both are orthogonal to the core spend circuit and can be layered on top.

10.8 Why Alice Cannot Steal Bob’s Note (Even Between Rooms)

Can *any* actor other than Bob – Alice-the-issuer, a prior holder, or a random Mallory – redeem `cm_0` under the shipped circuit? Two protections apply unconditionally; the third (full `A`-flavor binding) lands when `spend_a.circom` ships:

1. **Opening secrecy.** `spend.circom` requires the full Poseidon-5 opening (`flavor`, `v`, `rho`, `idHash`, `predicate`) as a private witness. Without `rho_0` and `idHash_0` the circuit is unsatisfiable, so no one without the opening can produce a valid nullifier or proof. For Alice’s secrets, this is the load-bearing protection on bearer-flavor (B1) notes: she could in principle race the bearer to deposit, which is the civil-trust semantics of bearer cash and is discussed at length in alberta-buck-notes.org §Long-Term Holding Safety.
2. **Recipient-binding at the payout site.** `recipient` is a public SNARK input ghost-bound by `recipient * recipient`, so Groth16’s IC[[commitment makes it non-malleable. A front-runner who intercepts Bob’s `proofBytes` and resubmits with a different `recipient` is rejected by the verifier. `chainId` is bound identically, blocking cross-chain replay.
3. **Recipient-binding in the nullifier (deferred).** In the full A-flavor design, the nullifier will be keyed on `M_rec` recovered from the in-circuit ElGamal decryption of `E_note` under `sk_dep`, with a Chaum-Pedersen equality against Bob’s registered credential proving `M_rec` equals Bob’s `M` (Theorem 8). That circuit is `spend_a.circom`, not yet shipped. Until then the shipped bearer-shape `spend` treats `idHash` as a generic witness scalar and Alice – who knows `idHash_0` and `rho_0` because she generated them at mint – could in principle redeem to herself. The wallet-level mitigation in the meantime is that `idHash` for A-flavor is a Poseidon hash of Bob’s identity that Alice binds at mint and discards immediately, leaving only Bob with the secret. The on-chain reinforcement is the pool’s `transferCarrying` recipient check, which the bearer-shape circuit does not bypass.

The *A-flavor strict-binding* guarantee Theorem 8 promises will become a chain-observable property once `spend_a.circom` is wired in. Today, the design intent for A-flavor still holds at the artifact level (Bob’s `idHash_0` is what authenticates him), and the SNARK enforces the bearer-shape discharge.

10.9 B-Flavor Notes Today

The shipped `spend.circom` is the B-shape circuit: Bob’s note above could equally well be a B1 bearer cheque, and the same five relation groups apply unchanged. Holder binding for B1 is the Schnorr-PoK style behaviour the wallet gets for free from the recipient-ghost-binding (relation 5) plus the depositor having to produce a valid `T_spend` from their own EOA: if two parties hold copies of the same B1 opening (e.g. a paper note photographed and forwarded), whichever submits `T_spend` first wins. That is the intended civil-trust semantics of a bearer cheque, not a bug. An explicit in-circuit Schnorr PoK over (`chainid || nullifier || recipient`) – to harden against opening-leakage between two B1 holders both submitting in the same block – is a possible follow-up but not in the shipped circuit.

10.10 Recapping Property C (Only Valid Notes Redeem)

The original Property C question – *subsequently only those N valid notes can be deposited* – becomes concrete at this layer. Claim: no actor can redeem a commitment `cm` unless

a. `cm` was appended to `commitments[]` by a prior `Notes.mint` transaction whose Groth16 proof verified (i.e., `cm` is part of an honest mint batch), and b. The actor holds the opening of `cm`, and c. For A-spends, the actor is the depositor keyed by `M_rec` in that opening.

Proof sketch. The `spend` circuit’s relation 2 (Merkle membership) rejects any `cm` not in `noteRoot`. The mint circuit’s relations 1-2 (opening binding and sum conservation) are what places a `cm` in

`noteRoot` in the first place, and by the mint-side soundness story in **Why Alice Cannot Cheat** every such `cm` corresponds to a well-formed opening whose `v` is included in the mint's `totalFace` escrow. The spend circuit's relation 5 (`face == v`) then ensures the BUCK released equals exactly the share of the escrow that `cm` was allocated. Summing over every accepted spend of a given mint batch, the cumulative BUCK released equals at most the `totalFace` escrowed – property C follows from mint-side value conservation plus spend-side nullifier uniqueness.

The earlier caveat about a missing `Num2Bits` range check on `v[i]` at mint time is closed: `mint_batch.circom` range-bounds each `v[i]` to 128 bits and `spend.circom` range-bounds both `face` and the witness `v` identically, so an integer overshoot of `totalFace` by a multiple of `r` is structurally impossible at either end. Property C now rests on those range bounds plus the nullifier-uniqueness check and Merkle membership.

11 Demurrage-Carrying Transfers

The Note-spend payout in Step 5 invoked `buck.transferCarrying`, a small sibling of the standard BUCK transfer that the `Notes` pool uses on its single egress. This section unpacks why one new method – not a per-account mode flag, not a routing decision, not a marker interface – is sufficient, and what its ledger effect looks like.

11.1 Two Ways to Handle Demurrage at Transfer Time

BUCK demurrage accumulates via a global `cumIndex` advancing at `BASE_RATE_PER_SEC` and a per-account `_indexAtLastTouch`. An account's accrued fee is the gap (`cumIndex - _indexAtLastTouch`) $\cdot$ `balance` at the moment the account is "touched" (standard transfer, settle, or carry). Two ledger responses to that accrued fee are distinguishable at transfer time:

- *Deducting transfer* (`transfer / transferFrom`, the default). On a 1,000-BUCK outgoing, Alice's accrued fee for the pre-transfer balance is settled at Alice – one-shot burned against her balance, resetting `_indexAtLastTouch[alice]` to the current `cumIndex`. The 1,000 BUCK then lands at Bob *fresh*: Bob's index is merged by a balance-weighted average toward `cumIndex`, diluting Bob's effective age. From a system perspective, accrued fees are burned at each transfer and total BUCK-age drops on Alice's side while the recipient enters fresh.
- *Carrying transfer* (`transferCarrying`, the new variant). On the same 1,000-BUCK outgoing, nothing settles at the sender: Alice's `_indexAtLastTouch` is unchanged, and her remaining balance keeps its age. Bob's index is merged by a balance-weighted average toward *Alice's* pre-transfer `_indexAtLastTouch`, so Bob inherits the age of the BUCKs he just received. Nominal balances move by exactly 1,000 on each side; no fee is burned; total system BUCK-age is preserved.

The invariant the carrying variant keeps is: **nominal BUCK out = nominal BUCK in** (no silent burn at the transfer site), with the recipient's forward index shift representing the demurrage they will pay on their next standard outgoing transfer.

11.2 Why the Note Pool Needs It

Three structural features of the pool make the deducting default wrong:

1. **Per-note age opacity.** When Alice mints six notes, the pool records only six opaque Poseidon commitments. The pool cannot deduct "the demurrage *Alice's specific note* owes" at spend time, because the chain does not distinguish one commitment's age from another. Any per-note fee would have to be computed off the pool aggregate, which would force Bob to pay fees that the pool's other depositors owed – and, worse, differentiate one commitment's payout from another, destroying the anonymity set.
2. **Face-to-payout identity.** `face` is the public SNARK output and the value the mint circuit committed to. A deducting pool transfer would deliver `face - fee` to Bob while the proof claimed `face`. This would break the mint-to-spend value-conservation invariant: the SNARK's promise would not match the ERC-20 ledger movement, and the spend circuit would have to become aware of demurrage accounting to compensate.
3. **Pool as bookkeeping buffer.** The pool is not a long-term BUCK holder; every BUCK that enters is earmarked for one future redeemer. Deducting fees at the pool's egress would siphon value into a burn that the pool has no way to replenish, producing a cumulative insolvency as outstanding notes circulate.

`transferCarrying` fixes all three: no per-note age, `face` to payout, no pool insolvency.

11.3 Shipped Surface in `Buck.sol`

The entire extension is one method plus one event:

```
event TransferCarrying(address indexed from, address indexed to, uint256 amount);

/// @notice Transfer 'amount' to 'to' carrying its accumulated BUCK-age. No fee
///         is burned at transfer time; recipient's index is shifted backward by
///         a balance-weighted average so total system age is preserved.
function transferCarrying(address to, uint256 amount) external returns (bool);
```

The preconditions match `transfer`: sender and recipient both `isVerified || isPublic` in the IdentityRegistry, recipient not the Jubilee address (the Jubilee is not an accepted carry destination). The weighted-merge update on the recipient is:

```
_indexAtLastTouch[to] =
    (br * _indexAtLastTouch[to] + amount * _indexAtLastTouch[from]) / (br + amount);
```

– the *sender's* current `_indexAtLastTouch` is the value merged in, not the global `cumIndex`. The sender's index is left untouched, and the fee burn path in `_update` is skipped via the settling flag so the carried merge survives.

There is deliberately **no per-account mode flag, no marker interface, no governance-gated whitelist, and no balance precondition**. Any registered account may invoke `transferCarrying`; it is a strictly additive method. What makes it pool-friendly is the *semantics* (no settle-at-sender, weighted-merge at recipient), not a policy toggle.

11.4 Who Else Might Call It

The Note pool is the first and canonical caller, but the method is not pool-private. Any registered account can invoke `transferCarrying` whenever carrying demurrage forward to the recipient is the right economic choice – escrow accounts whose beneficiaries are the true owners, payroll contracts pulling from an employer, cross-chain bridge egresses. The semantic invariant is "the source is a transitory holder of BUCK earmarked for the recipient," and in that case carrying the age forward matches the economic reality.

For direct peer-to-peer payments the deducting default remains correct: the sender is paying from their own pocket and is the natural place to burn their own accrued fee.

11.5 Open Question: Identity Receipts on Carry

The shipped `transferCarrying` uses the same `isVerified || isPublic` identity predicate as `transfer`; it does *not* require a Chaum-Pedersen approve receipt as `approve` does. Whether a full CP approval leg should be layered on for non-public senders is an open question; for the Note pool it is moot (the pool is `isPublic` by governance flag), but if `transferCarrying` is opened up to private EOAs an approve-style receipt may be wanted for auditability. See the **Open Work Items** in alberta-buck-ethereum.org.

12 Putting the Guarantees Side-by-Side

The mint walkthrough established five guarantees; the spend walkthrough adds four more that become concrete only once the redemption path exists.

Stage	Guarantee	Mechanism
Mint	Atomic conservation	Single EVM call: proof verify, <code>transferFrom</code> , append-cms all-or-nothing
Mint	Issuer-bound, recipient-blinded	<code>msg.sender</code> is public; <code>cm_i</code> is Poseidon-hidden over (<code>flavor</code> , <code>v</code> , <code>rho</code> , <code>idHash</code> , <code>pred</code>)
Mint	Per-note disclosure (recipient only)	Off-chain opening contains exactly one note's witness fields
Mint	Offline receipt reconstruction	<code>T_mint</code> calldata + opening + on-chain Identity lookup is sufficient and self-checkable
Mint	Adversarial-prover soundness	Groth16 knowledge soundness on fixed on-chain verifier; ERC-20 balance/allowance integr
Spend	Redemption unlinkable to specific <code>cm</code>	<code>cm</code> is a <i>private</i> witness under public <code>noteRoot</code> ; only nullifier + face + recipient revealed
Spend	Double-spend prevented	Deterministic nullifier + one-bit <code>nullifiers</code> mapping; rejected on reuse
Spend	Recipient-bound payout	<code>recipient</code> is a public SNARK input; proof rejects if re-targeted in-flight
Spend	Value conservation across <code>mint<->spend</code>	Range-bounded <code>mint v[i] + spend face == v=</code> ; cumulative payouts $\leq$ <code>totalFace</code> per bat

All nine rows are enforced by code in the tree. Mint landed in two phases: Phase 5 (`Notes.sol` with a stub verifier) and Phase 7 (per-leaf on-chain Poseidon insertion with `mint.circom`, `MintGroth16Verifier`, `MintVerifierAdapter`). Phase 7-bis moves the Merkle insertion *into* the SNARK (`mint_batch.circom`, per-N Groth16 verifiers, per-N adapter dispatch); the chain writes the SNARK-attested `newRoot` into the same 30-slot root- history ring `Notes.sol` already maintained for the spend side. Spend is unchanged by the pivot: `spend.circom` in bearer shape, `SpendGroth16Verifier`, `SpendVerifierAdapter`, the on-chain Merkle accumulator, and `bucket.transferCarrying`. The IdentityRegistry provides the issuer-attestation and recipient-verification legs, and the formal proofs (Theorems 7-11) provide the cryptographic backstop. The one residual caveat is that A-flavor in-circuit identity binding (Chaum-Pedersen equality on the depositor's registered credential) is deferred to `spend_a.circom`; until it ships, A-flavor privacy rests on the off-chain `idHash` secret and the pool's identity-bound transfer surface, not on an in-SNARK CP check.

13 Open Questions and Forward-Looking Caveats

The example above uses the Phase-7-bis batch-mint + Phase-7 spend pipeline. Six caveats remain and are tracked in depth elsewhere; listed here so the reader is not misled about the current state.

1. **Range bounds on $v[i]$ (resolved).** Earlier drafts flagged the possibility that `sum(v[i]) == totalFace` in `F_r` without per- v range checks would let an adversarial mint overshoot `totalFace` by a multiple of r . Both `mint_batch.circom` and `spend.circom` now range-bound each value via `Num2Bits(128)`. The integer-overflow attack is structurally impossible. Preserved here as a breadcrumb; see Open Question 1 in the proofs for the historical formulation.
2. **Implicit transfer $\leftrightarrow$ batch binding.** The chain emits `Transfer` and `Minted` in the same transaction, but no on-chain batch identifier ties the BUCK transfer to the specific `cm` range. Off-chain indexers reconstruct the binding from event correlation. Whether to add an explicit `batchId` storage slot (and a matching public input to the spend circuit) is an open design question; see Open Question 2.
3. **A-flavor in-circuit identity binding (deferred).** The shipped `spend.circom` enforces a bearer-shape (B-spend) discharge: knowledge of the Poseidon-5 opening is the spend authorization. Distinguishing A-flavor (in-circuit ElGamal decryption + Chaum-Pedersen equality against the depositor's registered credential) is deferred to `spend_a.circom`. Until that circuit is wired in, A-flavor notes rely for recipient binding on the secrecy of `idHash` in the off-chain artifact and on the pool's identity-bound `transferCarrying` recipient check, not on an in-SNARK CP proof. Theorem 8 in the proofs describes the intended binding; its chain-observable enforcement lands with `spend_a.circom`.
4. **Variable-N batch size (resolved via pinned set).** Phase-7's `mint.circom` was templated at $N=2$. Phase-7-bis generalises this: the wallet rounds any requested split up to the smallest pinned $N \geq N_{\text{requested}}$ from the deployed set (recommended $\{16, 128, 1024\}$), padding with $v = 0$ dummy openings. Each pinned N has its own verifier contract; the `MintVerifierAdapter` dispatches on `cms.length`. Adding a new pin means running a new trusted-setup ceremony, deploying a new verifier, and registering it in `mintVerifiers[N]` – operationally additive, no protocol change.
5. **Hash choice for `cmBatchHash` (open).** Two candidates: keccak256 (cheap on-chain, reconstructed from calldata in a few hundred gas) or Poseidon over the field (in-circuit "free" but $\sim 30\text{-}60\text{K}$ gas to recompute on-chain). Default is keccak; see the design memo for the trade-off. Revisit if an on-chain Poseidon precompile ships.
6. **Prover contention (resolved operationally, not protocolly).** The batch-mint pivot makes each mint a rollup-style transaction with a stale-state guard on `oldRoot` and `nextLeafIndex`. Losers in a race re-prove; no BUCK is lost. Initial deployment uses wallet-side optimistic retry; a sequencer/aggregator is deferred until real-world contention warrants it. See **Prover Contention and the Rollup Race** above.

14 Cross-References

- alberta-buck-notes.org – architectural overview of the Notes primitive (flavors, Merkle tree, demurrage interaction).

- alberta-buck-notes-rollup-mint.org – Phase-7-bis batch-mint design memo: R1CS counts per N, gas projections, per-attack-vector hostile-Alice audit (A1-A12), concurrency model, migration plan.
- alberta-buck-proofs.org – formal correctness arguments (Theorems 7-11; Open Questions in Part IV).
- alberta-buck-ethereum.org – Solidity surface, gas costs, phase-by-phase implementation status, Open Work Items.
- alberta-buck-identity.org – IdentityRegistry, PS credentials, and the bilateral ElGamal handshake that anchors `msg.sender` to a verifiable identity.
- `circuits/mint_batch.circom` – the Phase-7-bis batch-mint circuit (one compile per pinned N).
- `circuits/spend.circom` – the Phase-7 spend circuit (bearer-shape).
- `src/Notes.sol`, `src/MintGroth16Verifier*.sol`, `src/MintVerifierAdapter.sol`, `src/SpendGroth16Veri`
`src/SpendVerifierAdapter.sol` – the on-chain code paths walked above.
- `scripts/snark/prove_mint_batch.js`, `scripts/snark/prove_spend.js` – the reference provers used in tests.
- `test/MintVerifier.t.sol`, `test/SpendVerifier.t.sol` – the Foundry tests that pin the soundness stories to executable assertions.